

Lab 8: Privacy Preserving In Deep Learning

In this lab, we will train a deep learning model using Pytorch. We will also explore how differential privacy works in stochastic gradient descent (DP-SGD). After this lab, we expect students to know:

Objectives

1. PyTorch: What is PyTorch; How to install PyTorch;
2. Tensor: What is tensor; How to create tensor; How to use tensor;
3. MNIST: What is MNIST dataset; How does the dataset look like;
4. Deep Learning: How to build a neural network; How to train a model;
5. DP-SGD: How does DP-SGD work in deep learning; How does differential privacy affect model performance;

1. Pre-work: PyTorch

PyTorch is a Python-based scientific computing package that uses the power of graphics processing units. It is also one of the preferred deep learning research platforms built to provide maximum flexibility and speed. It is known for providing two of the most high-level features; namely, tensor computations with strong GPU acceleration support and building deep neural networks on a tape-based autograd systems.

PyTorch creators envisioned this library to be highly imperative which can allow them to run all the numerical computations quickly. This is an ideal methodology which fits perfectly with the Python programming style. It has allowed deep learning scientists, machine learning developers, and neural network debuggers to run and test part of the code in real time. Thus they don't have to wait for the entire code to be executed to check whether it works or not.

1.1. Installing PyTorch

Open a new terminal, put the following command:

```
$pip3 install torch
```

We also need Torchvision library, which is a part of Pytorch, contains all the important datasets as well as models and transformation operations generally used in the field of computer vision. It allows you to import datasets without any hassle. To install torchvision, we put the following command:

```
$pip3 install torchvision
```

1.2. Check the installation of PyTorch and Torchvision

To ensure we have successfully installed packages, we need the following command to check if packages are in the list:

```
$pip3 list
```

If packages are successfully installed, we can see their names and version numbers in the list.

In [1]:

```
1 from IPython.display import Image
2 Image(filename = "Desktop/list.png", width = 400, height = 100)
```

Out[1]:



```
torch          1.4.0
torchvision    0.5.0
```

1.3. Testing task: Build your first tensor

A tensor is a generic multiple-dimensional array, very similar to numpy arrays, or matrix. Tensors are used for arbitrary numeric computation. Now, let's build your first tensor. Create a new python file in Jupyter notebook, and type the following code:

In [2]:

```
1 import torch
2
3 x = torch.tensor([1,1,1,1,1])
4 y = torch.tensor([[1,1,1,1,1], [1,1,1,1,1]])
5
6 print("tensor x is:", x)
7 print("tensor y is:", y)
```

```
tensor x is: tensor([1, 1, 1, 1, 1])
tensor y is: tensor([[1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1]])
```

A tensor has a shape, to describe the number of dimensions and the length of each demension of this tensor. We can get the tensor's shape by:

In [3]:

```
1 print("tensor x's shape is:", x.shape)
2 print("tensor y's shape is:", y.shape)
```

```
tensor x's shape is: torch.Size([5])
tensor y's shape is: torch.Size([2, 5])
```

Tensors with the same shape can Participate in basic operations, e.g.

In [4]:

```
1 z = x + x
2 a = x * 2
3 b = x / x
4 print("New tensor z is:", z)
5 print("New tensor a is:", a)
6 print("New tensor a is:", b)
7 print(a==z)
```

```
New tensor z is: tensor([2, 2, 2, 2, 2])
New tensor a is: tensor([2, 2, 2, 2, 2])
New tensor a is: tensor([1, 1, 1, 1, 1])
tensor([True, True, True, True, True])
```

Practice: Basic arithmetic of tensors

In this practice, students are required to create tensors and operate basic arithmetic of tensors. We have the following tasks:

1. Create two tensors "a" and "b". tensor "a" is (2,4,6,8,10), and tensor "b" is (2,2,2,2,2).
2. Create a new tensor "c" which is equal to the sum of tensors "a" and "b". ($c = a + b$)
3. Create a new tensor "d" which is equal to the difference of tensors "a" and "b". ($d = a - b$)
4. Create a new tensor "e" which is equal to the product of tensors "a" and "b". ($e = a * b$)
5. Create a new tensor "f" which is equal to the division of tensors "a" and "b". ($f = a / b$)

2. Build a neural network

2.1. MNIST Dataset

The MNIST database (Modified National Institute of Standards and Technology database) is a large database of handwritten digits that is commonly used for training various image processing systems. The MNIST database contains 60,000 training images and 10,000 testing images. **Training images** are used to training and optimizing the model, and **test images** are used to test the model performance.

We import and download MNIST dataset from torchvision.

In [5]:

```
1 import torch
2 from torch import nn
3 from torchvision.datasets import MNIST
4 import numpy as np
5 from torch.autograd import Variable
6 import matplotlib.pyplot as plt
7
8 # Download the training dataset
9 train_set = MNIST('./data', train=True, download=True)
10 # Download the test dataset
11 test_set = MNIST('./data', train=False, download=True)
```

Now, let's see how the image looks like. We select any number of training images, e.g. 7th (a digital number 3). We print its label and the result shows as "3". We can see a real written number "3" in the image.

In [6]:

```
1 # Show the first image of the dataset
2 a_data, a_label = train_set[7]
3 print(a_data)
4 print(a_label)
5
6 a_data.show() # x is a PIL.Image here
```

```
<PIL.Image.Image image mode=L size=28x28 at 0x1208362E8>
```

Now, we transform this image and store its pixel as a numpy array. We then print it using plot.

In [7]:

```
1 # Print the Image in jupyter with numpy
2 a_data = np.array(a_data, dtype = 'float32')
3 print(a_data)
4 plt.imshow(a_data, cmap = 'gray')
```

[illegible]

Now, we transform all images and store their pixel in numpy array for training inputs. All images's pixel is $28 * 28 = 784$, so we need to reshape them as a one-dimension vector used for the inputs of the neural network.

In [8]:

```
1 def get_data(x):
2     x = np.array(x, dtype='float32') / 255
3     x = (x - 0.5) / 0.5
4     x = x.reshape((-1,))
5     x = torch.from_numpy(x)
6     return x
7
8 train_set = MNIST('./data', train = True, transform = get_data, download = True)
9 test_set = MNIST('./data', train = False, transform = get_data, download = True)
10
11 a, a_label = train_set[7]
12 print(a.shape)
13 print(a_label)
```

```
torch.Size([784])
```

```
3
```

2.2 Set the batch size

We Use DataLoader to define a data iterator. It's very important to create this iterator because the network cannot read a large amount of data once. We need the iterator to divide all images into many batches with a defined batch size. In our case, we set training data's batch size as 64, and test data's batch size as 128.

In [9]:

```
1 from torch.utils.data import DataLoader
2 train_data = DataLoader(train_set, batch_size = 64, shuffle = True)
3 test_data = DataLoader(test_set, batch_size = 128, shuffle = False)
4
5 a, a_label = next(iter(train_data))
6
7 # Print a batch of data
8 print(a.shape)
9 print(a_label.shape)
```

```
torch.Size([64, 784])
```

```
torch.Size([64])
```

2.3 Build the first network

Now, we start building our first network. we use Sequential to define a neural network with four layers. We use linear style in our model. In each layer, the first number is the size of inputs, and the second number is the size of outputs. Please note that the next layer's inputs' size must be equal to the outputs' size of the previous layer.

In [10]:

```
1 net = nn.Sequential(  
2     # First layer, input size 784, output size 400  
3     nn.Linear(784, 400),  
4     nn.ReLU(),  
5     # Second layer, input size 400, output size 200  
6     nn.Linear(400, 200),  
7     nn.ReLU(),  
8     # Third layer, input size 200, output size 100  
9     nn.Linear(200, 100),  
10    nn.ReLU(),  
11    # Fourth layer, input size 100, output size 10  
12    nn.Linear(100, 10)  
13 )
```

After building a network, we need to set the loss function and optimizer. In this case, we use Cross Entropy Loss function as our loss function, and SGD as our optimizer. The definition of them is shown below.

In [11]:

```
1 criterion = nn.CrossEntropyLoss()  
2 optimizer = torch.optim.SGD(net.parameters(), 1e-1)
```

In [12]:

```
1 # Build lists to store training loss,  
2 # training accuracy, test loss and test accuracy  
3 losses = []  
4 acces = []  
5 eval_losses = []  
6 eval_acces = []
```

Now, we can start training. We need to set the number of training epochs. Deep learning model contains too much computation which may require a long time each epoch. In our case, each epoch may spend approximate 5 seconds.

We need an appropriate epoch number because a small epoch number may cause that the model has not reached the convergence. A big epoch number may waste computation resources and time because the model has already reached the convergence and the performance cannot get better using the same model and training data.

In [13]:

```
1 # We set training epochs here.  
2 epochs = 20
```

Start training now

In [14]:

```
1  for e in range(epochs):
2      # Training
3      train_loss = 0
4      train_acc = 0
5      net.train()
6      for im, label in train_data:
7          im = Variable(im)
8          label = Variable(label)
9          out = net(im)
10         loss = criterion(out, label)
11         optimizer.zero_grad()
12         loss.backward()
13         optimizer.step()
14         train_loss += loss.item()
15         _, pred = out.max(1)
16         num_correct = (pred==label).sum().item()
17         acc = num_correct / im.shape[0]
18         train_acc += acc
19     losses.append(train_loss / len(train_data))
20     acces.append(train_acc / len(train_data))
21
22     # Testing
23     eval_loss = 0
24     eval_acc = 0
25     net.eval()
26     for im, label in test_data:
27         im = Variable(im)
28         label = Variable(label)
29         out = net(im)
30         loss = criterion(out, label)
31         # Save loss
32         eval_loss += loss.item()
33         # Save accuracy
34         _, pred = out.max(1)
35         num_correct = (pred==label).sum().item()
36         acc = num_correct / im.shape[0]
37         eval_acc += acc
38     eval_losses.append(eval_loss / len(test_data))
39     eval_acces.append(eval_acc / len(test_data))
40     print('epoch:{}, Train Loss:{:.4f}, Train Acc:{:.4f}, Eval Loss:{:.4f}, Eval Acc:{:.4f}'.format(e, train_loss/len(train_data), train_acc/len(train_data), eval_loss/len(test_data), eval_acc/len(test_data)))
41
```

epoch:0, Train Loss:0.5249, Train Acc:0.8316, Eval Loss:0.2052, Eval Acc:0.9351

epoch:1, Train Loss:0.1677, Train Acc:0.9471, Eval Loss:0.2168, Eval Acc:0.9279

epoch:2, Train Loss:0.1205, Train Acc:0.9630, Eval Loss:0.0952, Eval Acc:0.9688

epoch:3, Train Loss:0.0928, Train Acc:0.9710, Eval Loss:0.0916, Eval Acc:0.9706

epoch:4, Train Loss:0.0814, Train Acc:0.9741, Eval Loss:0.0852, Eval Acc:0.9721

epoch:5, Train Loss:0.0643, Train Acc:0.9794, Eval Loss:0.0717, Eval Acc:0.9774

epoch:6, Train Loss:0.0554, Train Acc:0.9823, Eval Loss:0.1006, Eval Acc:0.9675

epoch:7, Train Loss:0.0455, Train Acc:0.9851, Eval Loss:0.0770, Eval Acc:0.9761

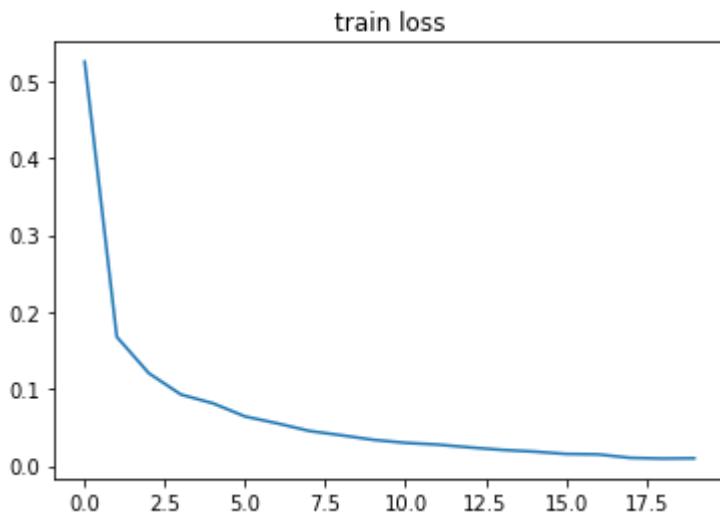
epoch:8, Train Loss:0.0399, Train Acc:0.9873, Eval Loss:0.0860, Eval Acc:0.9761

```
cc:0.9740
epoch:9, Train Loss:0.0340, Train Acc:0.9889, Eval Loss:0.0740, Eval A
cc:0.9760
epoch:10, Train Loss:0.0301, Train Acc:0.9899, Eval Loss:0.0958, Eval
Acc:0.9732
epoch:11, Train Loss:0.0279, Train Acc:0.9913, Eval Loss:0.1071, Eval
Acc:0.9676
epoch:12, Train Loss:0.0242, Train Acc:0.9920, Eval Loss:0.0690, Eval
Acc:0.9812
epoch:13, Train Loss:0.0210, Train Acc:0.9930, Eval Loss:0.0640, Eval
Acc:0.9811
epoch:14, Train Loss:0.0188, Train Acc:0.9939, Eval Loss:0.0644, Eval
Acc:0.9815
epoch:15, Train Loss:0.0157, Train Acc:0.9949, Eval Loss:0.0663, Eval
Acc:0.9818
epoch:16, Train Loss:0.0151, Train Acc:0.9951, Eval Loss:0.0673, Eval
Acc:0.9815
epoch:17, Train Loss:0.0106, Train Acc:0.9965, Eval Loss:0.0657, Eval
Acc:0.9814
epoch:18, Train Loss:0.0097, Train Acc:0.9971, Eval Loss:0.1184, Eval
Acc:0.9637
epoch:19, Train Loss:0.0100, Train Acc:0.9972, Eval Loss:0.0691, Eval
Acc:0.9815
```

Now, we plot the training loss, training accuracy, test loss and test accuracy in axis.

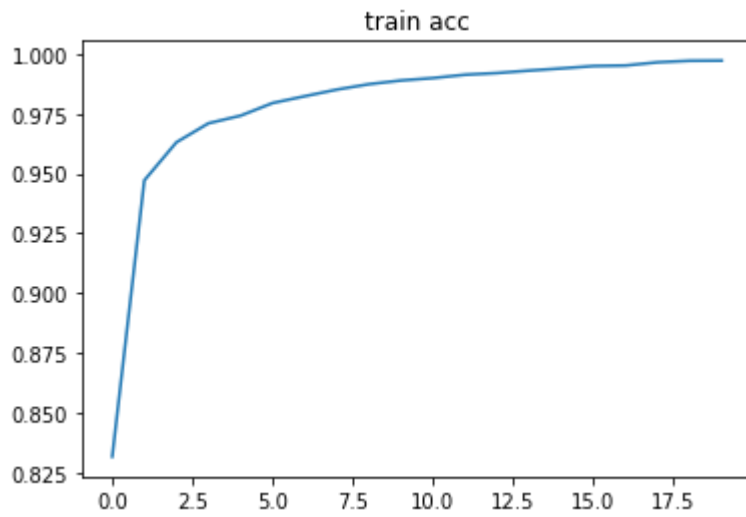
In [15]:

```
1 # Print training loss
2 plt.title('train loss')
3 plt.plot(np.arange(len(losses)), losses)
4 plt.show()
```



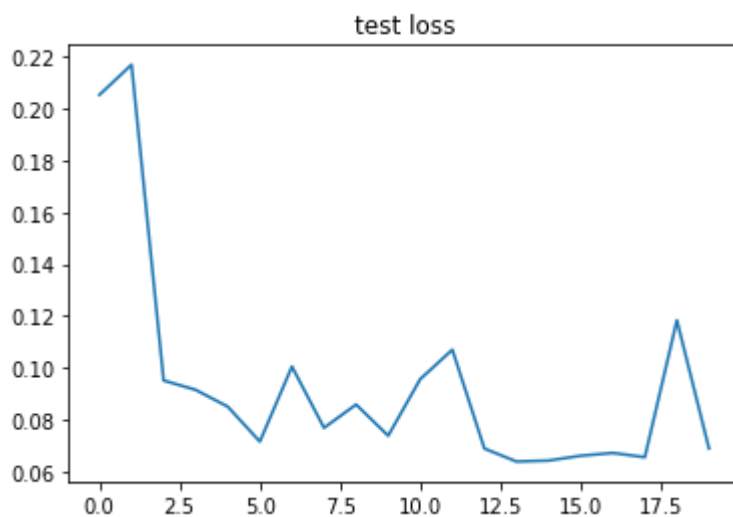
In [16]:

```
1 # Print training accuracy
2 plt.plot(np.arange(len(accs)), accs)
3 plt.title('train acc')
4 plt.show()
```



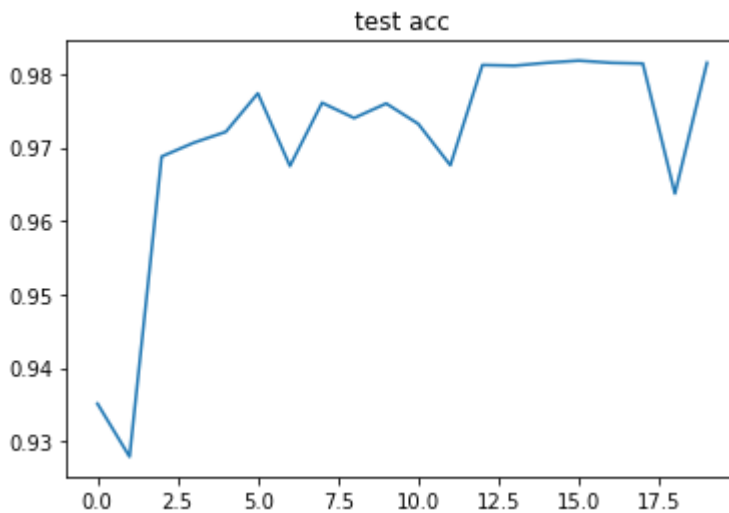
In [17]:

```
1 # Print test loss
2 plt.plot(np.arange(len(eval_losses)), eval_losses)
3 plt.title('test loss')
4 plt.show()
```



In [18]:

```
1 # Print test accuracy
2 plt.plot(np.arange(len(eval_acces)), eval_acces)
3 plt.title('test acc')
4 plt.show()
```



Now, we will explore the impact of differential privacy on the model's performance. DPSGD combines differential privacy mechanism and SGD together as an optimizer. We will use DPSGD instead of SGD. Please make sure you have download DPSGD.py file with the same root of this file.

In [19]:

```
1 # Build a new network named "net1".
2 # The name of this net must be different with the last one.
3 net1 = nn.Sequential(
4     # First layer, input size 784, output size 400
5     nn.Linear(784, 400),
6     nn.ReLU(),
7     # Second layer, input size 400, output size 200
8     nn.Linear(400, 200),
9     nn.ReLU(),
10    # Third layer, input size 200, output size 100
11    nn.Linear(200, 100),
12    nn.ReLU(),
13    # Fourth layer, input size 100, output size 10
14    nn.Linear(100, 10)
15 )
16
17 # Define loss function
18 import DPSGD
19 criterion = nn.CrossEntropyLoss()
20 # Define optimizer. We use DPSGD here, which combines differential privacy mecha
21 optimizer = DPSGD.DPSGD(net1.parameters(), lr=1e-1, batch_size=32, C=1.0, noise_mult
```

In [20]:

```
1  # Start training
2  losses1 = []
3  acces1 = []
4  eval_losses1 = []
5  eval_acces1 = []
6
7  epochs = 20
8
9  for e in range(epochs):
10     train_loss = 0
11     train_acc = 0
12     net1.train()
13     for im, label in train_data:
14         im = Variable(im)
15         label = Variable(label)
16         # Forward
17         out = net1(im)
18         loss = criterion(out, label)
19         # Backward
20         optimizer.zero_grad()
21         loss.backward()
22         optimizer.step()
23         # Training loss calculation
24         train_loss += loss.item()
25         #
26         _, pred = out.max(1)
27         num_correct = (pred == label).sum().item()
28         acc = num_correct / im.shape[0]
29         train_acc += acc
30     losses1.append(train_loss / len(train_data))
31     acces1.append(train_acc / len(train_data))
32     # Test
33     eval_loss = 0
34     eval_acc = 0
35     net.eval()
36     for im, label in test_data:
37         im = Variable(im)
38         label = Variable(label)
39         out = net1(im)
40         loss = criterion(out, label)
41         # Test loss calculation
42         eval_loss += loss.item()
43         # Test accuracy
44         _, pred = out.max(1)
45         num_correct = (pred==label).sum().item()
46         acc = num_correct / im.shape[0]
47         eval_acc += acc
48     eval_losses1.append(eval_loss / len(test_data))
49     eval_acces1.append(eval_acc / len(test_data))
50     print('epoch:{}, Train Loss:{:.4f}, Train Acc:{:.4f}, Eval Loss:{:.4f}, Eval
51         .format(e, train_loss/len(train_data), train_acc/len(train_data), eval_loss/
```

epoch:0, Train Loss:2.1975, Train Acc:0.3353, Eval Loss:1.9487, Eval A
cc:0.5485

epoch:1, Train Loss:1.4766, Train Acc:0.6639, Eval Loss:1.1104, Eval A
cc:0.7373

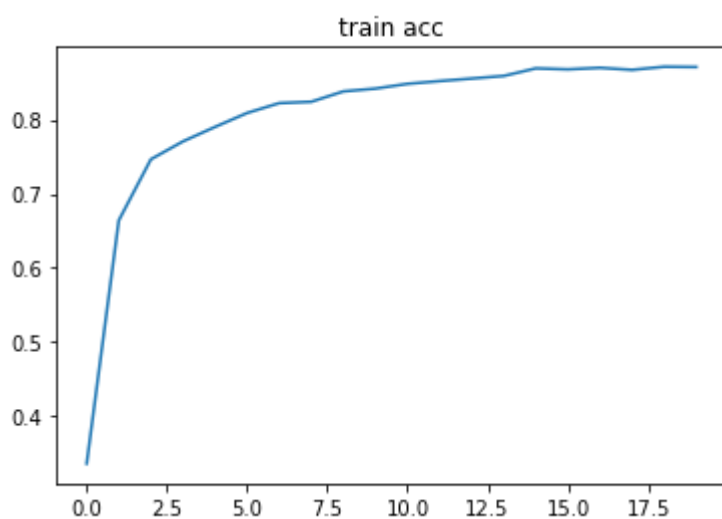
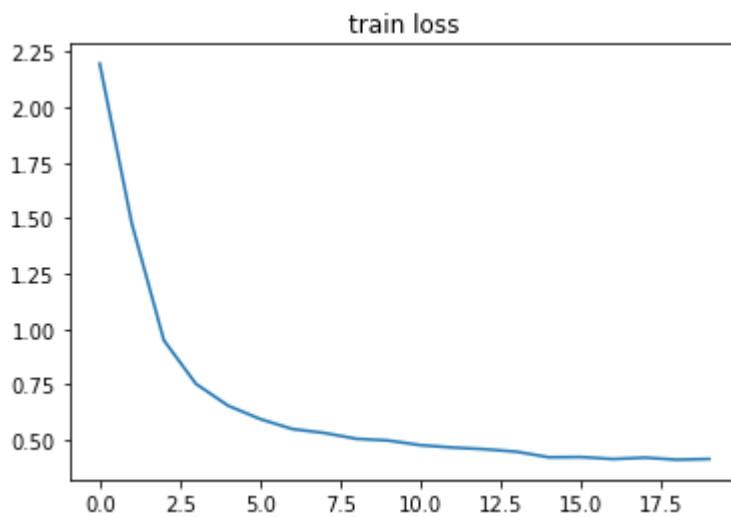
epoch:2, Train Loss:0.9496, Train Acc:0.7465, Eval Loss:0.8223, Eval A
cc:0.7878

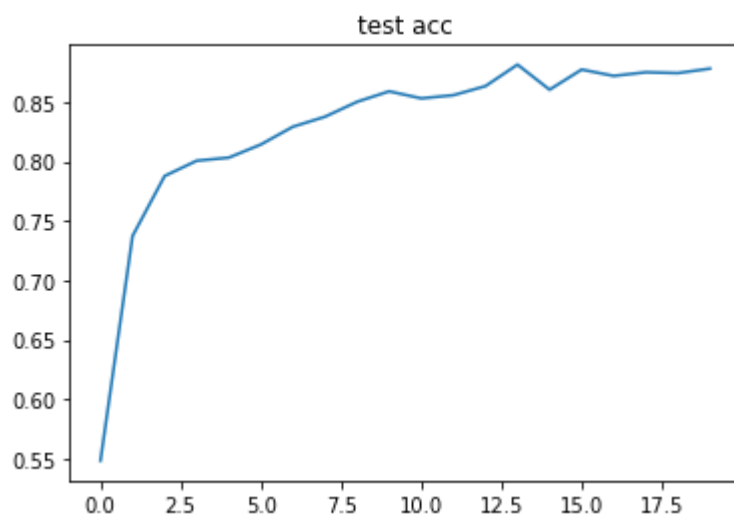
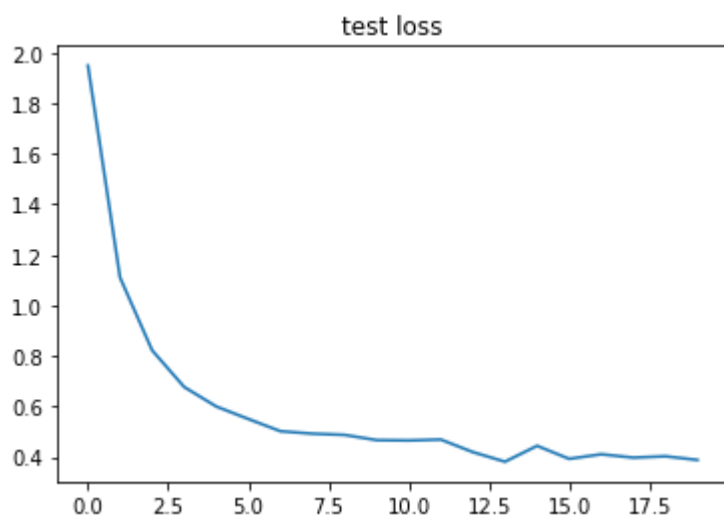
epoch:3, Train Loss:0.7533, Train Acc:0.7707, Eval Loss:0.6772, Eval A

cc:0.8006
epoch:4, Train Loss:0.6550, Train Acc:0.7903, Eval Loss:0.6001, Eval A
cc:0.8032
epoch:5, Train Loss:0.5947, Train Acc:0.8091, Eval Loss:0.5507, Eval A
cc:0.8143
epoch:6, Train Loss:0.5497, Train Acc:0.8224, Eval Loss:0.5022, Eval A
cc:0.8292
epoch:7, Train Loss:0.5319, Train Acc:0.8243, Eval Loss:0.4925, Eval A
cc:0.8376
epoch:8, Train Loss:0.5056, Train Acc:0.8383, Eval Loss:0.4877, Eval A
cc:0.8502
epoch:9, Train Loss:0.4977, Train Acc:0.8421, Eval Loss:0.4670, Eval A
cc:0.8590
epoch:10, Train Loss:0.4767, Train Acc:0.8485, Eval Loss:0.4657, Eval
Acc:0.8530
epoch:11, Train Loss:0.4661, Train Acc:0.8522, Eval Loss:0.4693, Eval
Acc:0.8557
epoch:12, Train Loss:0.4587, Train Acc:0.8557, Eval Loss:0.4195, Eval
Acc:0.8633
epoch:13, Train Loss:0.4471, Train Acc:0.8592, Eval Loss:0.3817, Eval
Acc:0.8814
epoch:14, Train Loss:0.4222, Train Acc:0.8695, Eval Loss:0.4445, Eval
Acc:0.8604
epoch:15, Train Loss:0.4233, Train Acc:0.8681, Eval Loss:0.3929, Eval
Acc:0.8773
epoch:16, Train Loss:0.4142, Train Acc:0.8702, Eval Loss:0.4111, Eval
Acc:0.8719
epoch:17, Train Loss:0.4205, Train Acc:0.8674, Eval Loss:0.3978, Eval
Acc:0.8750
epoch:18, Train Loss:0.4113, Train Acc:0.8716, Eval Loss:0.4031, Eval
Acc:0.8743
epoch:19, Train Loss:0.4146, Train Acc:0.8713, Eval Loss:0.3886, Eval
Acc:0.8781

In [21]:

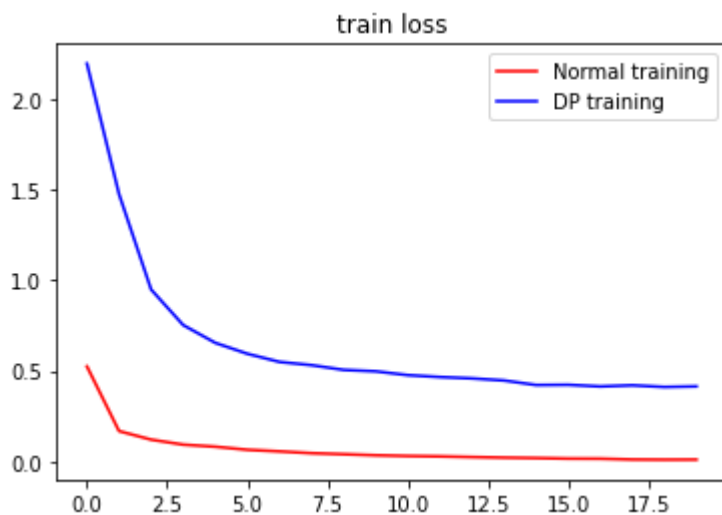
```
1 # Print training loss
2 plt.title('train loss')
3 plt.plot(np.arange(len(losses1)), losses1)
4 plt.show()
5
6 # Print training accuracy
7 plt.plot(np.arange(len(accs1)), accs1)
8 plt.title('train acc')
9 plt.show()
10
11 # Print test loss
12 plt.plot(np.arange(len(eval_losses1)), eval_losses1)
13 plt.title('test loss')
14 plt.show()
15
16 # Print test accuracy
17 plt.plot(np.arange(len(eval_accs1)), eval_accs1)
18 plt.title('test acc')
19 plt.show()
```

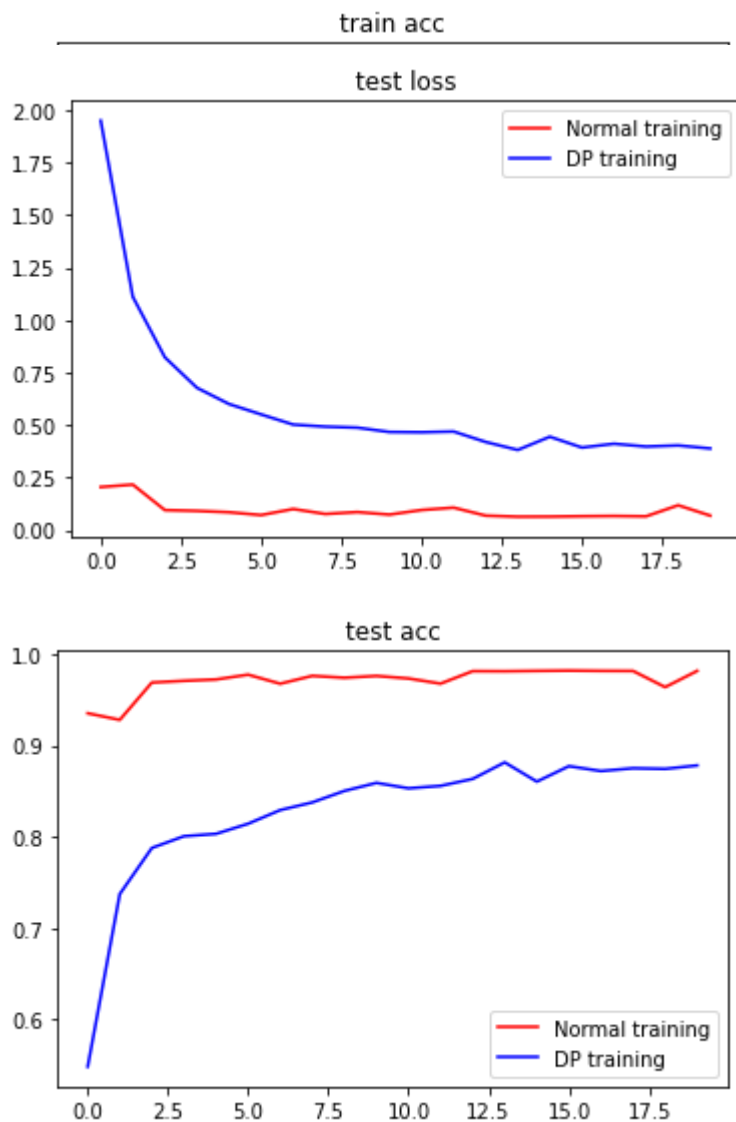




In [26]:

```
1 # Print training loss
2 plt.title('train loss')
3 plt.plot(np.arange(len(losses1)), [losses[i] for i in range(len(losses))], color = 'red')
4 plt.plot(np.arange(len(losses1)), [losses1[i] for i in range(len(losses))], color = 'blue')
5 plt.legend()
6 plt.show()
7
8 # Print training accuracy
9 plt.plot(np.arange(len(acces1)), [acces[i] for i in range(len(acces1))], color = 'red')
10 plt.plot(np.arange(len(acces1)), [acces1[i] for i in range(len(acces1))], color = 'blue')
11 plt.legend()
12 plt.title('train acc')
13 plt.show()
14
15 # Print test loss
16 plt.plot(np.arange(len(eval_losses1)), [eval_losses[i] for i in range(len(eval_losses1))], color = 'red')
17 plt.plot(np.arange(len(eval_losses1)), [eval_losses1[i] for i in range(len(eval_losses1))], color = 'blue')
18 plt.legend()
19 plt.title('test loss')
20 plt.show()
21
22 # Print test accuracy
23 plt.plot(np.arange(len(eval_acces1)), [eval_acces[i] for i in range(len(eval_acces1))], color = 'red')
24 plt.plot(np.arange(len(eval_acces1)), [eval_acces1[i] for i in range(len(eval_acces1))], color = 'blue')
25 plt.legend()
26 plt.title('test acc')
27 plt.show()
```





Conclusion

In this lab, we have learnt:

1. defining tensors;
2. basic operations of tensors;
3. MNIST dataset
4. deep learning and neural network;
5. DP-SGD in deep learning

From the training results we can know that:

1. the noise from differential privacy (DP-SGD) may slightly reduce the accuracy of model.
2. the size of noise is affected by sensitivity and privacy budget.
3. it is important to balance the privacy and model utility.